Source Control and Databricks

Last updated by | Gupta Vishal kailash | Apr 11, 2019 at 2:48 PM GMT+1

Databricks CLI Preparation

1. Download and Install Python.
   https://www.python.org/downloads/
2. Once installed, make sure you have path environment variable set for Python scripts
3. Now run the following command
   ```
   pip install databricks-cli
   ```
4. To connect to Databricks Workspace using Databricks-CLI you'll require a personal Databricks Token
   https://docs.databricks.com/api/latest/authentication.html
5. Run the following command to configure connection to Databricks Workspace using the token:
   ```
   databricks configure --token
   ```
   You'll be prompted for the Databricks Host, which in our case is https://northeurope.azuredatabricks.net/?o=5628103188513068 and your personal token

Please note that the configuring databricks creates a new file called **.databrickscfg** in your current working directory. This directory contains the Databricks workspace URL as well as your token.
Ideally you will be running databricks CLI commands from different folders. To enable that, add a environment variable called DATABRICKS_CONFIG_FILE (include the filename in the path).
When the token expires, you'll be able to replace it with a new one in the .databrickscfg file (or run the configure command again).

You can now use databricks CLI with Databricks Workspace.

GIT CLI Preparation

1. Install git. You may want to use the following link.
   https://git-scm.com/download/win
2. Run the following commands for some basic configurations
   ```
   git config --global user.name "<Your name>"
   git config --global user.email "<Your email address>"
   ```

You can now use git CLI to manage repositories.

Databricks Workspace and Git

Given that we have a shared Databricks Workspace and all developers are going to share it, we need a process for code management. As for all projects, you begin with cloning the repository to your laptop/PC. Open a command prompt, navigate to the area where you want to create the clone and then run the following command to clone Ingestion-Attunity repository. Change the URI for appropriate repository as necessary.

`git clone https://anglianwater.visualstudio.com/Data%20Services/_git/Ingestion-Attunity`

I have identified that the https://dev.azure.com/anglianwater/... version of the URL doesn't work with git.

The command `git clone` allows you to clone the default branch of the repository to your pc. If you want to place the clone in a directory other than your working directory, just add the destination path at the end of the above command. For all repositories in DevOps, we have the 'development' branch as the default branch. You should be required to do this step only once for a repository.

Every time a code change is required, a new feature branch should be created from the 'development' branch. One way of doing it is from DevOps portal. After creating a feature branch, you should do a pull request in your clone folder for it to know that a new branch has been created. Use `git pull` to do that.

Next, navigate within the clone folder and run the following command to checkout your feature branch.

```
git checkout <Feature Branch>
```

Now before importing anything into Databricks Workspace, delete everything you have within your own AWBI folder within the workspace.

```
databricks workspace delete -r /Users/<Your username>@anglianwater.co.uk/AWBI/
```

Run the following command to deploy the feature branch under your personal area within Databricks workspace. The following example is for deploying rawAttunity. Use similar commands for other repositories.

```
databricks workspace import_dir -o ./rawAttunity /Users/<Your username>@anglianwater.co.uk/AWBI/rawAttunity
```

At this point you'd have:

- Cloned the repository
- Created a feature branch for the task you're working on
- Switched to the feature branch
- Have the project deployed in your personal area within Databricks workspace

At this point the developer makes appropriate changes in the code within Databricks workspace. After completing the changes, the following steps will help checking the code into the repository.

Delete the contents of your clone folder (except the .git folder and readme.md file)

Run the following command to get the latest in the clone directory.

```
databricks workspace export_dir /Users/<Your username>@anglianwater.co.uk/AWBI/rawAttunity ./rawAttunity
```

If you want to identify or verify the files that have changes, run the following:

```
git status
```

The files that have changed will be in shown in red.

Then you'll need the following git commands to add (stage), create commit, and push.

```
git add .
git commit -m "Latest changes in AWBI Databricks"
git push
```

Please note the files would only have been committed in the feature branch by this point. Go to the browser now and create a pull into development.

Housekeeping

Ideally you'd like to keep your local clone synced with the development branch. To do that, switch to the development branch, delete the feature branch and pull the latest changes from development branch. Here are the commands to do that:

```
git checkout development
git branch -d <Feature Branch>
git pull
```